

CS151 Intro to Data Structures

Java Basics

Data Structures

What you'll learn:

1. Data Structures
2. Programming and Debugging Skills
3. Designing Complex Programs

Today

Part 1:

- Introductions
- Administrative info
- Syllabus
- Tips for Success

Part 2:

- Java Basics
- Exercises:
 - Ex 1: classes, arrays, loops
 - Ex 2: input / output, exceptions

Administrivia

- Course website
 - BMC-CS-151.github.io
 - Assignments and lab instructions, syllabus
 - Code from lecture
- Piazza:
 - Asynchronous communication
 - Can post anonymously (anonymous just to classmates)
 - Answer your peers questions!
 - Counts for participation grade
- Gradescope:
 - Entry code **8YG6VP**
 - Submit all assignments
 - Can request re-grade requests
 - WHAT YOU SEE IS WHAT YOU GET
- Optional Textbook
- 2.7 GPA requirement for CS Major

Schedule

- Lecture Tues / Thursday
- Homeworks due on Fridays at 11:59pm
 - Two late days allowed
 - On Sunday at 11:59pm, the submission window will be closed.
- Lab Park 231/Thursday 2:40pm-4:00pm
 - Instructor: Prof. Dianna Xu
- Midterm: Thursday October 22nd in class
- Final Exam: During finals period. Date TBD

Grade Breakdown

- Homeworks: 20%
- Labs: 10%
- Quizzes: 25%
- Midterm: 15%
- Final: 30%

Homeworks

- All submitted on Gradescope and autograded

Average Workload

(reported by past students)

HW0: 6 hours

HW1: 11 hours

HW2: 19 hours

HW3: 7 hours

HW4: 6 hours

HW5: 13 hours

HW6: 16 hours

HW7: 20 hours

HW8: 15 hours

How to succeed

1. DO YOUR HOMEWORK
2. Start early
3. Ask for help
 - a. Piazza
 - b. TA and Professor office hours

Tips for Success

- Get things working in the smallest case and continue to iterate on assignment until the deadline
- Compile early and often
- Prioritize homeworks
- START EARLY

Policies

<https://bmc-cs-151.github.io/policies.html>

- **Support:** Attend office hours and ask questions on Piazza. For additional support, visit the Q-Center and the Office of Academic Support.
- **Attendance:** Expected at all lectures/labs; participation can tip borderline grades. All quiz and exam dates for the first half of the course are posted on the website. Report conflicts within the first two weeks.
- **Late work:** 2-day grace period after the deadline with no penalty; after that, extensions require emailing both instructors with documentation.
 - It can be difficult to estimate how long an assignment will take. Start assignments early and work incrementally. As you are coding, compile early and often.
 - There is no limit to the number of times you can submit to gradescope.
 - Continue submitting partial solutions to incrementally earn points. A submission that does not compile on gradescope will not earn any correctness points (up to 95% of the overall grade).
 - Be strategic in your work such that you can earn partial credit if the assignment is not fully completed by the deadline.
- **Collaboration:** Discussing ideas is fine, but no sharing code/answers; work must be your own.
- **Accommodations:** Contact instructors early with a verification letter.

AI / Outside Source Policy

Grades in this course are largely quiz/exam based (70%). The assignments are meant to assist you in learning the material and practicing your programming skills. Having a strong understanding and mastery of the material will give you the best chance to perform well on the exam, and beyond. This course is foundational for a CS career, both at Bryn Mawr and beyond. Learning the content and building your skills in this course are crucial to investing in yourself as a computer scientist.

Understanding a given solution is very far away from being able to come up with one yourself. The following are guidelines to avoid shortcuts:

- When possible, use course resources (class examples, textbook, and Oracle Documentation) rather than googling. LLMs are probabilistic models and can lie / hallucinate API information.
- When using AI and outside sources (internet) you should avoid asking questions that are overly contextualized to your own code. Try to ask general questions and avoid sharing code you wrote with any online tools.
- Write your own code. Copy-pasting code from an LLM or outside source is detrimental to your learning. Writing your own code and working through compiler and runtime errors refines your skills, and promotes learning. To be an effective computer scientist, you should be comfortable on the terminal, writing code, and working through errors. The best way to build these skills is to practice them.

Prerequisites

- Comfortable in command line and vim
 - If you're not - complete Lab0
- Java
 - basic syntax - we'll review some today

Your Environment

- CS server account
 - Make sure you can log in
 - Email David Diaz if encountering issues (ddiaz1@brynmawr.edu)
- Lab00: ideally completed before Lab Thursday, getting up and running with vim and linux
- Software: vim, Java, or just ssh

Part 2: Java Basics and CS1 Review

Java for the Python Developer

<https://bmc-cs-151.github.io/study-guide.html>

Compilation: Java is a compiled language. Unlike `.py` files, `.java` files must be compiled before they can be executed.

For example given a file `HW1.java`,

```
javac Hw1.java
```

creates a `.class` file which can be run with:

```
java Hw1
```

The compiler is your friend. It will tell you when there are errors. Compile early and often.

Java for the Python Developer

Hierarchical Structure: In python you can freely declare variables or write print statements at any level. In java this is not allowed. Everything exists within a class. Statements/Expressions must be inside methods which must be inside classes.

In python you can just write,

```
def f(x):  
    return x + 1
```

In java this must exist inside a class:

```
class Utils {  
    int f(int x) {  
        return x + 1;  
    }  
}
```

Notice that the method `f` has an explicit return type (`int`) and parameter type (also `int`). This brings us to our next major difference between python and java...

Java for the Python Developer

Static Typing: All variables must be declared with *type* of data they are storing.

Method parameters and return types also must be explicitly declared and are enforced by the compiler.

Types.java

CS1 Review Topics

1. Classes - accessors, constructors, this keyword, new keyword, toString, object equality
2. Arrays - initialization, default values, searching through an array
3. Command Line Arguments
4. Scanner - reading from user input and reading from a file
5. Exceptions

Classes and Objects

A class is a way to organize data. It is a blueprint.

An Object is an instance of a class

Create objects by calling the constructor with the `new` keyword

The constructor is a special method that is called to create an object

Arrays

An array is a fixed-size ordered collection of data

Ways to declare an array:

- `int[] nums = new int[5];`
 - `nums[0] = 100;`
- `int[] nums = {100, 200, 300, 400, 500};`

How would I declare a 2D array (3x3) of Strings?

Access the size of the array using `nums.length;`

Exercise 1 -

Part a: Create a College class with:

name,

number of students,

year founded

Part b: In the main, create 3 colleges and put them in an array

Part c: Take a college name as input and print the year it was founded

User Input

There are two ways to read in data that we will use in this course. The major difference is when the data is supplied.

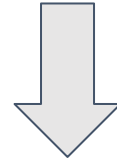
For command line arguments the data is supplied directly when you run the program.

For example: `java Add 5 7`

With the **Scanner**, we can take input while the program is running. For example:

```
$ java Add  
Please supply which numbers you would like to add  
5  
7
```

User Input



```
Scanner input = new Scanner(System.in);

System.out.print("Enter your name: ");
String name = input.nextLine();    // reads a whole line

System.out.print("Enter your age: ");
int age = input.nextInt();         // reads a single int

System.out.print("Enter your GPA: ");
double gpa = input.nextDouble();  // reads a single double
```


Exercise 2 - count words in a file

Part a: Read in a filename from command line

Part b: Count the number of words in the file

Exceptions – way to deal with unexpected events during execution

- Unexpected events:
 - unavailable resource
 - unexpected input
 - NPE
 - AOB

How do we deal with exceptions?

```
try {  
    guardedBody  
} catch (exceptionType1 variable1) {  
    remedyBody1  
} catch (exceptionType2 variable2) {  
    remedyBody2  
} ...  
...
```

Pop Quiz!

Does not count for your grade

Summary

- Lab 1 Thursday
- HW0 Released. Due Friday Sep 11th
 - Start early!
- Join Piazza and Gradescope